Centro de Informática UFPE

Feature Modelling

Enzo Gurgel Bissoli(egb2)

UFPE CIn

2026-03-30

Principles of Feature Modeling

Abstract

-> Provide systematic understanding of the system

-> main challenges: **identify**, group and organize features

```
featureModel :: Set Feature -> Config -> Either InvalidConfig Product
```

-> Do principles to create, evolve and mantain featureModel exist? that's the value proposition of the paper

-> Phases of feature modelling

```
type FeatureModellingPhases = Planning | Construction | Maintenance | Evolution
```

Introduction

-> problem: integrate feature that may have many depedencies considering all possible product variants

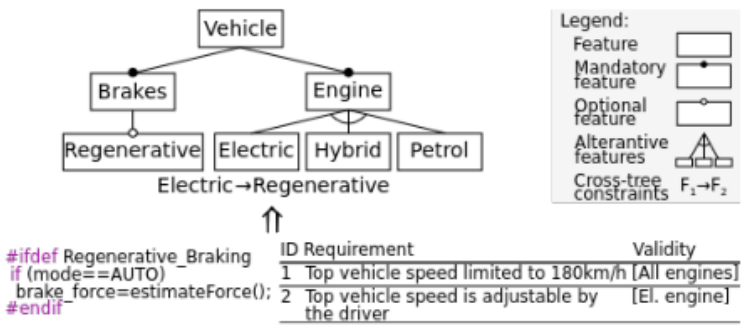
(+) limited knowledge on how to design in practice

(?) Complex management -> modelling

-> what: a notation that uses tree like structure for features that can derive product variants (not necessarily valid ones though)

-> how: concepts available (optional/mandatory features, feature hierarchy, feature groups, cross-tree constraints)

(+) extended by vendors with (feature attributes, cardinalities or non propositional constraints)



-> validation: systematic literature review, 31 papers, ten interviews with experts that applied feature modelling in real life projects

Background and Motivation

Centro de Informática UFPE

-> when: feature driven development, SCRUM/XP

-> What: constrainsts of the form parent->child implications, feature typing, mostly boolean values, feature groups

(+) crosstree constraints are propositional expressions

-> how: experience reports -> distinct artifacts -> comparison -> diffs -> feature model

Methodology

```
principleMaking :: SnowBallMethod -> Identification -> Interviews -> Analysis -> Triangulation -> Syntethis
```

Identified Principles

Planning and preparation

Centro de Informática UFPE

-> Identify the relevant stakeholders

(+) sources, producers and consumers

-> In immature or heterogeneous domains, unify the terminology

(+) domain dictionary as precondition for success

-> Define the purpose of the feature model

(+) product scoping vs product development (what that even means?)

(-) “they show an example where feature modelling capture the problem space, solution space and the configuration space”

(+) reduce ambiguity and wasted drawing boards

-> Define criteria for feature to sub-feature decomposition

(-) hard to identify relationship and responsibilities between features

(+) provides a how to decompose features

(++) low number of cross-tree constraints, requires heavy prototyping, hierarchy represents the natural decomposition

-> Plan feature modelling as an iterative process

(+) alternate between domain scoping and modelling, never a single task, but both

(++) “train the domain expert with simpler models, practice in low stake environments”

-> Keep the number of modellers low

(+) recommended to be a single person!

Training

Centro de Informática UFPE

-> Familiarize with the basics of product-line engineering

(+) map concepts in real life to notation using concepts such as binding time, variability, product lines and so on

-> Select a small sub-system to be modeled for training

(+) less moving parts, easier to reason about and extract the abstraction without complexity, lot's of commonality in relation to final product
arbitrary fast feedback-loops

-> Conduct a pilot project

(.) last close to three days

(.) guide stakeholders in distinguishing distinctions between domain concepts and implementation level concepts

Information Sources

Centro de Informática UFPE

-> Rely on domain knowledge and existing artifacts to construct the feature model

(.) require domain experts and stakeholders to provide information and feature identification

(+) aggregate the knowledge in “enginnering arctifacts” such as specifications, vocabularies, manuals and so on

(++) “we look at source code clones/branches/versions to get the product diiffirences and identify and extract features from these distinctions manually”

Model organization

Centro de Informática UFPE

-> The depth of the feature-model hierachy should not exceed eight levels

(+) between 3 to six level deeps, remember is not a binary tree, but a unbalanced m-ary tree

(++) More hierarchy ? Split the model, cue to modularization

(+++) The linux kernel has staggering **8 level deep nodes**

-> features at higher levels in the hierachy should be more abstract

(+) abstraction should flow from the top to bottom

(++) middle level are functional aspects of the feature

(++) bottom levels are detailed features

(+++) a distinction between **architectural** and **implementation** feature

-> Split larger modules and facilitate consistency with interface models

(.) modularity is a well know principle in software engineering and appears everwhere

(+) how to ensure consistency?

(++) interface models: features that are inter-model depedent, (bound contract)

-> Avoid complex cross-tree constraints

(-) hampers comprehension

(+) concept of “presence conditions” or “tagging”

-> Maximize cohesion and minimize coupling with feature groups

(+) this is engineering 101

Modelling

Centro de Informática UFPE

-> Use workshops to extract domain knowledge

(+) most efficient method to extract domain knowledge

(++) “Why does the difference exist” is the trigger question to build understanding

-> Focus first on identifying features that distinguish variants

(++) much easier to make variants obvious than commonalities clear

-> Apply bottom-up modelling to identify distinctions between artifacts

-> Apply top-down modelling to identify distinctions in domains

(+) good for commonalities

-> Use a combination of bottom-up and top-down modelling

(+) top down -> abstract view

(+) bottom up -> fine grained “implementation” view

(++) both increase overall completeness of the model!

-> A feature typically represents a distinctive, functional abstraction

(+) features should be used to capture **externally visible characteristics**, don’t forget the granularity!

-> If needed, introduce spurious features

(.) I guess that means introduce a default value for features when a consensus can't be reached

(+) a feature that server as dummy for other features, or to make some constraint possible

-> Define default feature values

(+) what would be "reconfiguration"?

(++) concept of visibility conditions

-> Define feature-model views

Partition the model(s) as required for the proper level of abstraction(value?) be conveyed

(+) profiles a way to “expose” a subset of a feature model to another (maybe for interface models?)

-> Prefer Boolean type features for comprenhension

(+) “nice way of organizing configuration switches”

-> Document the features and the obtained feature model

(+) Every new terminology must be available to the stakeholders asap, so jit docs is a must

(++) documentation and implementation must be simultaneous activities

what about infra as code principles?

Dependencies

Centro de Informática UFPE

-> If the models are configured by (company) experts, avoid feature dependency modeling

(-) dependency identification is very expensive

(+) avoid or model simple dependencies

-> If the main users of a feature model are end-users, perform feature dependency modeling

(.) because the paying customer is the one using the model, so make your product robust

(+) reduce conflict resolution

(+) choice propagation

Quality Assurance

-> Validate the obtained feature model in workshops with domain experts

(+) introduce multiple domain experts when validating, note validation is not modeling

(+) right way to name things and structure them

-> Use the obtained feature model to derive configurations

(+) being able to derive configurations that are not implemented, but valid products, show the coverage factor of a feature model

(+) if the current feature model lacks new derivations than the analyzed ones, the product line is probably a high risk investment

-> Use regression tests to ensure that changes to the feature model preserve previous configurations

(+) creating reference variants (what that entails?) reduce the probability of model drift

Model Maintenance and Evolution

Centro de Informática UFPE

-> Use centralized feature model governance

(+) Centralize access to the model so that drift is minimized

-> Version the feature model in its entirety

(+) reduce inconsistencies and allow rollbacks

-> New Features should be defined and approved by domain experts

(+) new features require: specification -> definiton -> integration/ simulation (High risk!)

Discussion

-> properties of the principles

(+) multiple perspectives on the same issue (views vs feature splits)

(+) principles are highly coupled to the who of modeling

(+) principles are bound to scenarios, that is hard to find commonalities, but they do exist

(.) Most planning and training principles are vendors or industrial cases (not the general set of practitioners)

(+) notation usage is domain independent

Related Work

-> FORM methodology as an increment to FODA method

Threats to validity

-> Interviews could have been biased, here the authors mention the sources of bias

-> Construct Validity

(+) intro to the interviewee project, terminology adaptation

-> Internal validity

(+) Ensure domain independence principles (broad interviewee experience)

-> External validity

(+) Some principles are context sensitive

-> Conclusion validity

(+) Principles are general and applicable to a broader scope

(+) Study focus on reproducible methods (so maybe Methodology is reproducible)

Conclusion

-> 34 atemporal principles

-> 8 feature modeling categories

-> Interview with experts in the domain